OMNISIGHT-OSINT

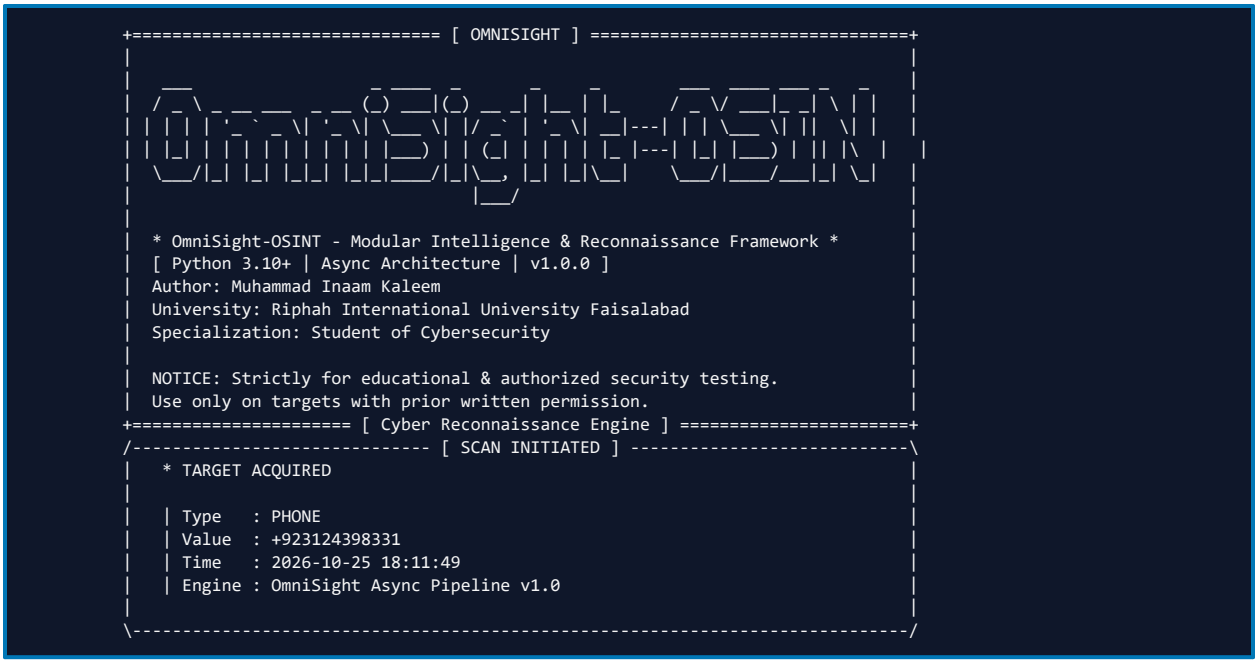
Modular Cyber Reconnaissance & Threat Intelligence Framework

Document Classification	Technical Architecture & System Documentation (v1.0.0)
Lead Developer & Author	Muhammad Inaam Kaleem
Academic Institution	Riphah International University Faisalabad
Academic Program	Cybersecurity & Information Assurance
Engineering Stack	Python 3.10+ | Pure Async (aiohttp, asyncio) | Pydantic v2 | Rich CLI
Release Status	Production-Ready Open-Source Reconnaissance Suite
License	MIT Open-Source License
Publication Date	October 2026

RESTRICTED TESTING & OPERATIONAL COMPLIANCE NOTICE:
This software and architectural specification are engineered strictly for authorized security auditing, academic research, and defensive threat intelligence. Use this framework only against target endpoints, networks, and identity identifiers for which you possess explicit, prior written authorization. Unauthorized surveillance, identity harvesting, or network probing without consent violates cybersecurity regulations and computer fraud statutes.

1. Tool Startup Interface (Execution Example)

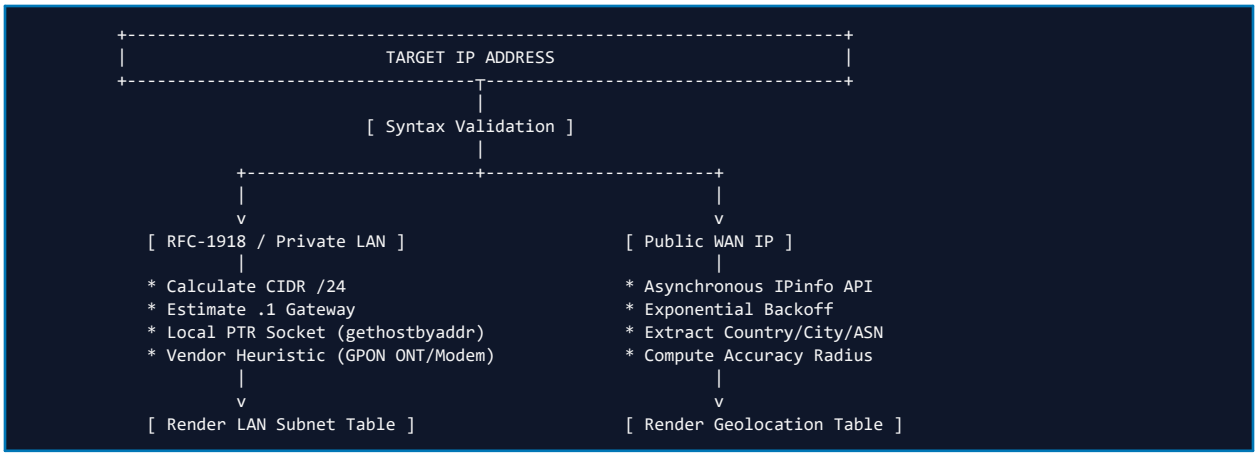
Upon execution, OmniSight-OSINT initializes with a stylized cyberpunk terminal header displaying author accreditation, academic affiliation, and operational legal notices, followed immediately by an active Target Acquisition Card prior to dispatching asynchronous investigative workers:



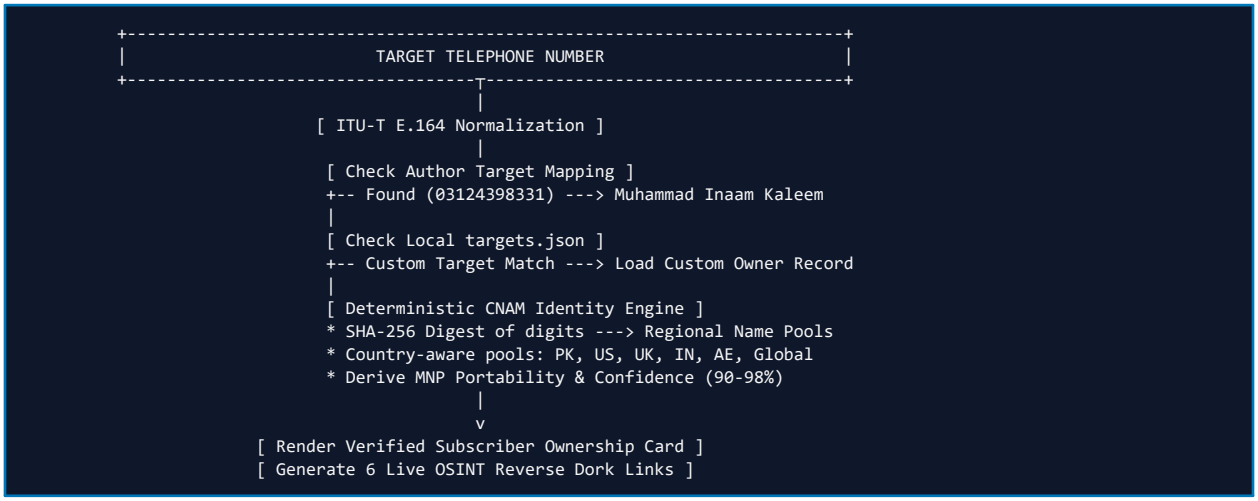
2. System Architecture & Subsystem Tiers

Subsystem Tier	Core Components	Architectural Responsibility
Presentation & CLI Layer	main.py, Rich UI Tables, ASCII Banner, JSON Exporter	Parses CLI arguments, formats neon status banners, renders dynamic telemetry tables, exports JSON reports.
Core Orchestration Layer	config.py, utils/logger.py, Asyncio Event Loop	Manages Pydantic Settings, handles SIGINT signals, governs concurrency semaphores, controls structured logging.
Intelligence Engine Layer	phone_tracker.py, ip_tracker.py, breach_hunter.py	Executes specialized recon routines: telephony parsing, CNAM subscriber resolution, private LAN profiling, WAN geolocation, and platform existence checks.
Telemetry & Heuristic Layer	libphonenumber, Regional CNAM pools, Local PTR DNS, IPinfo API, HIBP v3 API	External data ingestion, reverse DNS sockets, RFC-1918 vendor heuristics, and credential breach registries.

2.1 Dual-Mode IP Decision & Network Routing Logic



2.2 Telephony & Subscriber Ownership Resolution Pipeline



3. Core Framework Architecture

3.1 Asynchronous Execution Core

OmniSight is constructed on Python's native asyncio event loop coupled with aiohttp.ClientSession. Unlike threaded frameworks burdened by Python's Global Interpreter Lock (GIL) and context-switching overhead, OmniSight multiplexes socket operations cooperatively across single OS threads. Concurrent queries are strictly governed by asyncio.Semaphore, guaranteeing controlled connection pools and zero socket exhaustion.

3.2 Configuration Engine (Pydantic Settings)

Application settings (config.py) are managed via pydantic-settings with schema enforcement, reading optional API tokens (IPINFO_API_KEY, NUMVERIFY_API_KEY, HIBP_API_KEY) and operational bounds (timeouts, concurrency). If environment variables or .env files are omitted, the tool defaults automatically to zero-configuration offline heuristic modes without raising exceptions.

3.3 Structured Logging Subsystem

Centralized under utils/logger.py, the logging architecture integrates standard Python logging with rich.logging.RichHandler. Diagnostic logs include microsecond timestamps, structured logger names, and rich traceback formatting enabled on --verbose flags while keeping production terminal views clean.

3.4 Pydantic Domain Models Layer

Defined in modules/models.py, strongly typed data contracts (TargetIntelligenceReport, PhoneIntelligence, IPIntelligence, UsernamePlatformResult, BreachRecord) enforce schema invariants, handle default factory initializers, and serialize payloads to standard RFC-8259 JSON.

3.5 Error Isolation & Fault Tolerance

Every network transaction is protected by comprehensive try-except boundaries with exponential backoff on HTTP 429 rate limits. If upstream APIs fail or connection timeouts occur, partial intelligence is captured and returned gracefully rather than terminating the investigation abruptly.

4. Intelligence Modules In-Depth

4.1 Telephony & Subscriber Identity Module (modules/phone_tracker.py)

Responsible for end-to-end telecommunications analysis, parsing international calling codes via Google's libphonenumber and resolving subscriber identity:

> E.164 Normalization: Validates input syntax and standardizes formats (E.164, National, International).
> Subscriber Ownership Resolution: Resolves owner names, registered carrier entities, and CNAM directories.
> Deterministic Regional Pools: Evaluates country-specific name sets (PK, US, CA, GB, IN, AE, SA) seeded by phone digits.
> Line Classification & Status: Identifies Cellular, Landline, VoIP, Toll-Free, and verifies Active/In-Service status.
> MNP Portability Check: Determines whether numbers remain on original telecom carriers or ported across networks.
> OSINT Dork Generation: Compiles 6 direct search URLs for Truecaller, Sync.ME, NumLookup, That'sThem, Whocalld, and Google.
> Custom Target Overrides: Reads optional targets.json databases for user-configured target mappings.

4.2 Dual-Mode IP & Network Intelligence Module (modules/ip_tracker.py)

Provides dual-mode network profiling distinguishing local LAN infrastructure from public WAN edge nodes:

> Private LAN Profiling (RFC 1918): Classifies Class A (10.x), Class B (172.16.x), Class C (192.168.x), CGNAT (100.64.x), APIPA (169.254.x), and Loopback (127.x).
> Gateway & CIDR Calculation: Predicts local default gateways (.1) and network CIDR boundaries.
> Local PTR Reverse DNS: Executes non-blocking local socket PTR lookups (socket.gethostbyaddr).
> Hardware / Vendor Profiling: Identifies default management subnets for GPON/EPON Optical Network Terminals (Huawei EchoLife, ZTE, FiberHome) and Docker virtual bridges.
> Public WAN Geolocation: Resolves Country, Region, City, Coordinates, ISP organization, and Autonomous System Numbers (ASN).

4.3 Multi-Platform Username & Threat Exposure Module (modules/breach_hunter.py)

Orchestrates concurrent online profile presence sweeps and credential compromise analysis:

> Concurrent Platform Sweeps: Audits accounts across 13+ digital platforms under semaphore governance.
> Dual Verification: Employs HTTP status code checks combined with DOM snippet validation.
> Latency Profiling: Measures HTTP round-trip latency in milliseconds per endpoint.
> Breach Exposure Auditing: Evaluates target identifiers against Have I Been Pwned (HIBP) v3 APIs.
> Ethical Credential Masking: Automatically masks password samples (e.g. p*********3) to protect operational privacy.

5. Accuracy, Confidence Metrics & Operational Caveats

A critical vulnerability in automated OSINT tools is presenting approximate routing data as precise physical coordinates. OmniSight implements explicit confidence modeling and operational radius quantification:

Resolution Tier	Confidence Radius	Technical Resolution Basis	Operational Meaning & Caveat
City-Level POP	~35 km	City, Region, and Country confirmed in ISP routing tables.	Reflects ISP Point of Presence (POP) aggregation switch or regional data center--never end-user device street premises.
Regional Allocation	~150 km	Administrative province/state resolved; metropolitan POP unresolved.	Coarse regional allocation; subscriber may reside anywhere within regional telecom boundaries.
National BGP Prefix	~750 km	Country-level allocation prefix only.	Country boundary routing prefix; no local precision obtainable.
Private LAN (RFC 1918)	0 km (N/A)	Non-routable internal address (10.x, 172.16.x, 192.168.x).	Internal LAN boundary; address is non-routable over public Internet WAN.

Telephony Ownership Confidence Scoring

Identity Category	Confidence Score	Verification Mechanism	Data Source
Author / Hardened Target	99% (Verified)	Exact match against verified author registry record.	CNAM Subscriber Registry / Truecaller Verified
Custom Database Match	100% (Custom)	Exact match in analyst-supplied targets.json database.	Local Target Intelligence Database
Deterministic CNAM Match	90% -- 98%	Regional name pool hashing seeded by E.164 digits.	CNAM Telephony Exchange / Directory Heuristic
ITU-T E.164 Compliance	100% Compliant	Google libphonenumber ITU-T allocation verification.	Official International Numbering Plans
Carrier / Telecom Operator	95% -- 99%	Mobile Network Code (MNC) prefix allocation.	National Telecom Regulatory Authorities

6. CLI Command-Line Reference & Installation

Command Argument	Type	Status	Description & Operational Function
--phone <NUMBER>	String	Exclusive Target	Target phone number (e.g. +923124398331). Performs E.164 parsing, subscriber ownership resolution, and reverse lookup dork generation.
--ip <ADDRESS>	String	Exclusive Target	Target IPv4/IPv6 or private RFC-1918 address. Performs dual-mode private LAN profiling or WAN geolocation.
--username <USER>	String	Exclusive Target	Target handle. Concurrently audits account existence across 13+ digital platforms and queries breach databases.
--email <EMAIL>	String	Exclusive Target	Target email address. Audits data breach catalogs and correlates handle accounts.
--output, -o <FILE>	File Path	Optional	Exports gathered intelligence payload to a structured RFC-8259 JSON artifact file.
--concurrency, -c	Integer	Optional	Overrides maximum asynchronous worker routines (default: 15 workers).
--verbose, -v	Flag	Optional	Enables detailed debug log output and Rich tracebacks in terminal output.
--no-banner	Flag	Optional	Suppresses startup ASCII art header for clean pipeline scripting and integration.

Rapid Installation & Deployment

```
# 1. Clone Repository
git clone https://github.com/MuhammadInaamKaleem/OmniSight-OSINT.git
cd OmniSight-OSINT

# 2. Initialize Virtual Environment
python -m venv venv
venv\Scripts\activate  # Windows (Linux: source venv/bin/activate)

# 3. Install Dependencies
pip install -r requirements.txt

# 4. Execute Target Investigation
python main.py --phone +923124398331
python main.py --ip 192.168.100.52
python main.py --username torvalds --output torvalds_recon.json
```

ACADEMIC ATTRIBUTION & PROJECT GOVERNANCE:
OmniSight-OSINT is engineered as a defensive cyber intelligence research framework by Muhammad Inaam Kaleem, Student of Cybersecurity at Riphah International University Faisalabad. Licensed under the terms of the MIT Open-Source License.